Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
| Wydział Inżynierii Mechanicznej i Robotyki | Katedra Robotyki i Mechatroniki

INŻYNIERIA MECHATRONICZNA | I stopień | semestr VI | 2025/2026

Programowanie Obiektowe i Inżynieria Oprogramowania

Instrukcja C++/CLI:
3. Integracja GUI z warstwą logiczną – dolewanie substancji

Wprowadzenie do aplikacji okienkowych w C++

Nauczysz się:
Pisać kod pod aplikacje okienkowe w języku C++/CLI z wykorzystaniem platformy *.NET framework* w środowisku *Visual Studio*:

- poznasz nowe kontrolki, jak Button i ComboBox,
- zwizualizujesz stan obiektów zaimplementowanych w języku C++ (*backend*) w GUI stworzonym w C++/CLI.

Dodatkowe materiały:

- materiały do zajęć: [link],
- oficjalna strona C++/CLI [link].

Kierunkowe efekty kształcenia pokryte w tej instrukcji:
IME1A_U1, IME1A_U4

Koordynator przedmiotu:
dr inż. Lucjan Miękina, miekina@agh.edu.pl

Autor instrukcji:
dr inż. Jakub Bryła, jakub.bryla@agh.edu.pl

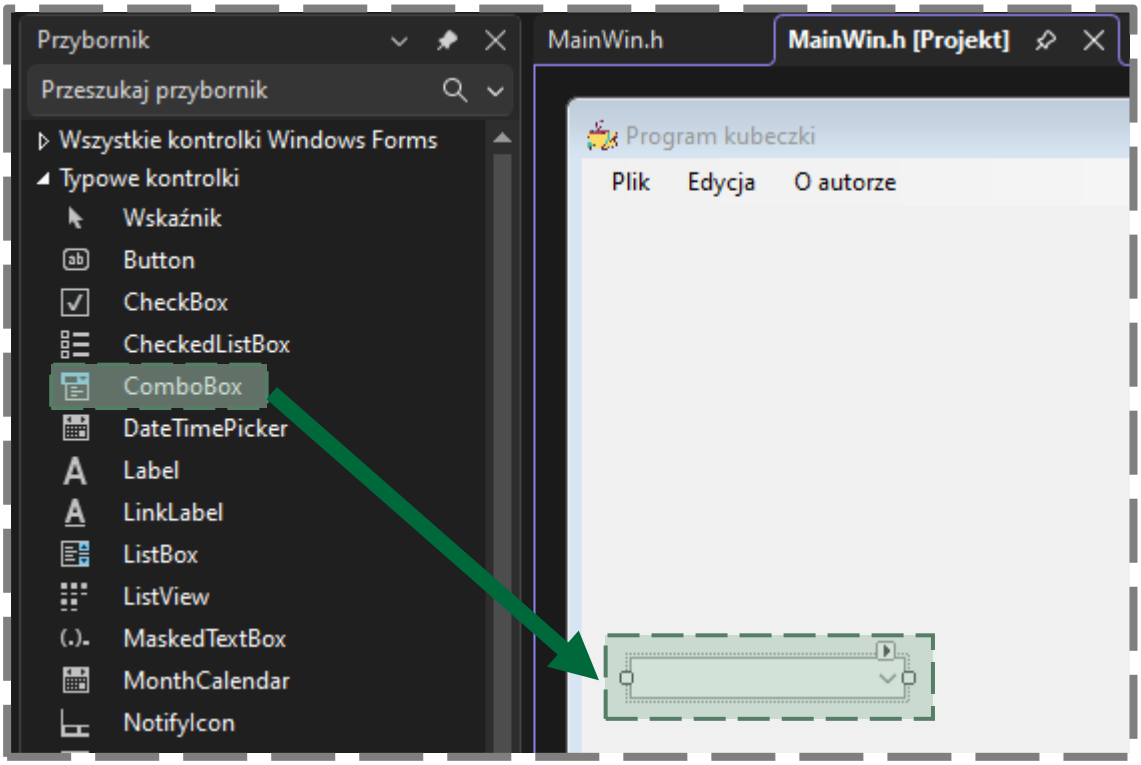
1. Wprowadzenie

Ostatnia instrukcja zakończyła się dodaniem do projektu klas *TCup* i *TSubstance*. Kompilacja zakończyła się bez błędów, jednak w aplikacji okienkowej nie widać efektu kodu warstwy logicznej… W tej instrukcji, poza dodaniem kilku nowych kontrolek GUI, skupimy się na integracji GUI z *backendem*.

2. Zadania

ZADANIE 2.1 – dodawanie substancji do kubka: wyświetlenie kontrolek GUI

Zaimplementujemy kod, który spowoduje wyświetlenie się listy substancji, poniżej etykiety aktywnego kubka, po wybraniu opcji w menu „Dodaj substancje do kubka”. W pierwszej kolejności musimy dodać kontrolkę ***ComboBox*** do głównego okna aplikacji:



Zmieniamy wybrane wartości w oknie ***Właściwości***:

3. Nazwa kontrolki (*Name*): „*subList*”,
4. Pozycja początkowa (*Location*): „*12; 273*”,
5. Wymiar (*Size*): „*199; 21*”,
6. Brak możliwość wpisywania własnych wartości (*DropDownStyle*): „*DropDownList*”,
7. Widoczność (*Visible*): „*False*”.

Dodajemy obsługę zdarzenia: kliknięcia lewym przyciskiem myszy (LPM) na opcję menu „Dodaj substancje do kubka”. W tym celu klikamy dwukrotnie LPM na wspomnianą opcję w zakładce „*MainWin.h [Projekt]*” – do wygenerowanego szkieletu funkcji wstawiamy kod:

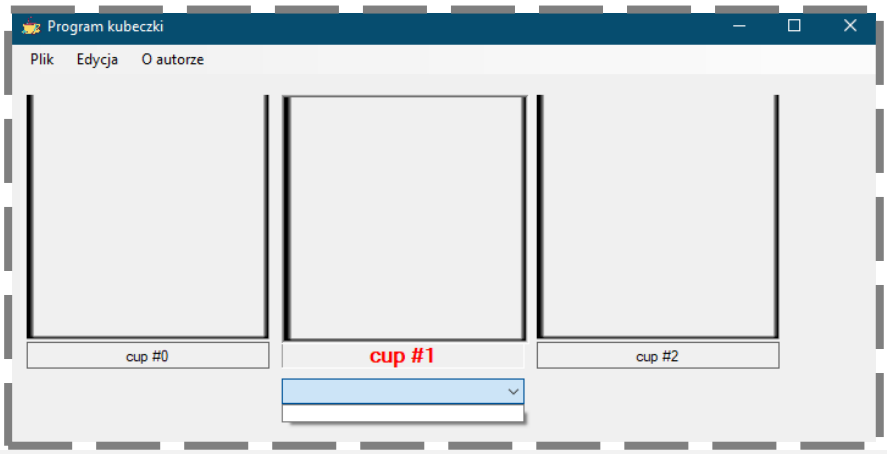
```
private: System::Void addSubMenuItem_Click(System::Object^ sender, System::EventArgs^ e) {
        subList->Location = System::Drawing::Point(12 + (10 + 199) * cupID, 273);
        subList->Visible = true;
}

```

W przypadku dezaktywacji kubka musimy zadbać o schowanie kontrolki ***ComboBox*** – dodajemy instrukcję na końcu funkcji *cleanLblCup*():

```
private: Void cleanLblCup() {
        for each (Label ^ lbl in lbl_cups)
        {
        lbl->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
        lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8, System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
                lbl->ForeColor = System::Drawing::Color::FromArgb(0, 0, 0);
        }

        for each (PictureBox ^ pb in cups)
        {
                pb->BorderStyle = System::Windows::Forms::BorderStyle::None;
        }

        delCupMenuItem->Enabled = false;
        delCupMenuItem->Text = L"Usuń kubek";
        addSubMenuItem->Enabled = false;
        addSubMenuItem->Text = L"Dodaj substancje";

        subList->Visible = false;
        cupID = -1;
}
```

Uruchom aplikację i sprawdź jej działanie:

1. utwórz 3 kubki,
2. aktywuj jeden z nich,
3. wybierz opcję „Dodaj substancję do kubka”:



4. aktywuj inny kubek i wybierz opcję menu „Dodaj substancję do kubka” – sprawdź czy kontrolka ***ComboBox*** zmieniła poprawnie lokalizację,

5. kliknij w pusty obszar okna aplikacji – żaden kubek nie jest aktywny, kontrolka ***ComboBox*** powinna zniknąć.

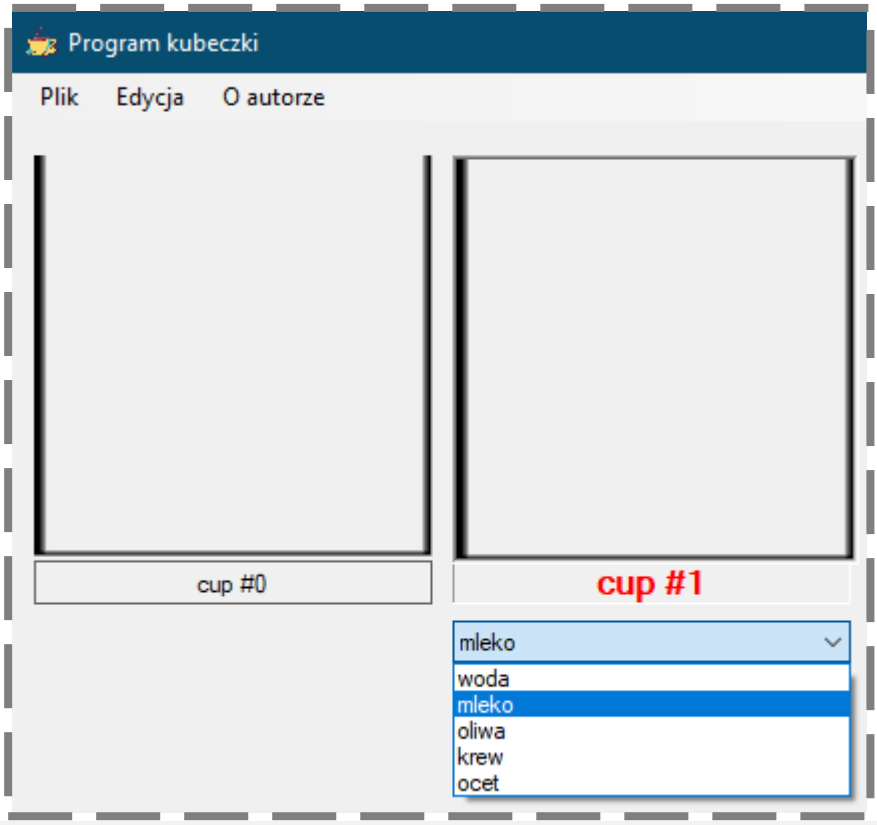
Jeśli wszystko jest w porządku, przejdźmy do uzupełnienia listy substancji. Utwórzmy funkcję pomocniczą poniżej funkcji *getCupID()*:

```
private: Void fillSubList() {
        subList->Items->Clear();
        int count = substancje.size();

        for (int i = 0; i < count; i++) {
                std::string name = substancje[i].get_name();
                String^ name_cli = gcnew String(name.c_str());
                subList->Items->Add(name_cli);
        }
}

```

I wywołajmy ją w funkcji obsługującej zdarzenie kliknięcia w opcję menu „Dodaj substancję do kubka”:

```
private: System::Void addSubMenuItem_Click(System::Object^ sender,
System::EventArgs^ e) {
        fillSubList();
        subList->Location = System::Drawing::Point(12 + (10 + 199) * cupID,
273);
        subList->Visible = true;
}

```

Efekt działania nowego kodu:



Dodajmy kolejne kontrolki:

1. ***TextBox*** dla możliwości wpisania ilości dolewanych ml cieczy,
2. dwa przyciski ***Button***: „*Wlej*” i „*Anuluj*”.

Dla ***TextBox*** zmień wartości w oknie ***Właściwości***:

1. *Name*: „*subMl*”,
2. *Location*: „*12; 302*”,
3. *Size*: „*199; 20*”,
4. *Text*: „*Wpisz ilość w [ml]*”,
5. *TextAlign*: „*Right*”,
6. *Visible*: „*False*”.

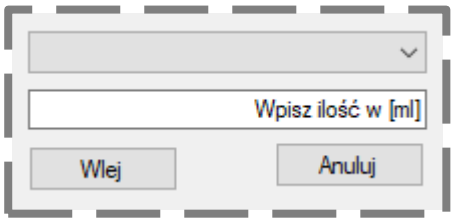
Dla ***Button*** „*Wlej*” zmień wartości w oknie ***Właściwości***:

1. *Name*: „*wlej*”,
2. *Text*: „*Wlej*”,
3. *Location*: „*12; 330*”,
4. *Visible*: „*False*”.

Dla ***Button*** „*Anuluj*” zmień wartości w oknie ***Właściwości***:

1. *Name*: „*anuluj*”,
2. *Text*: „*Anuluj*”,
3. *Location*: „*135; 330*”,
4. *Visible*: „*False*”.

Widok w zakładce „*MainWin.h [Projekt]*”:



Zajmijmy się obsługą kontrolki do wpisywania ilości wlewanej cieczy. Kliknij LPM na etykietę w oknie aplikacji (zakładka „*MainWin.h [Projekt]*”), przejdź do okna ***Właściwości***, zmień widok na obsługę zdarzeń i kliknij dwukrotnie LPM na akcję „*MouseDown*”. W wygenerowanym szkielecie funkcji wstaw kod:

```
1  private: System::Void subMl_MouseDown(System::Object^ sender,
   System::Windows::Forms::MouseEventArgs^ e) {
2          subMl->Text = "";
3  }
4
```

Działanie powyższej funkcji polega na usunięciu tekstu zachęty w momencie kliknięcia w obszar kontrolki. Musimy jeszcze sprawdzić czy wprowadzoną wartość można przekształcić na liczbę – będziemy to sprawdzać za każdym razem gdy klawisz zostanie wciśnięty, w momencie gdy kontrolka jest aktywna. W tym celu kliknij w oknie ***Właściwości*** dwukrotnie LPM na akcję „*KeyUp*”. W wygenerowanym szkielecie funkcji wstaw kod:

```
1  private: System::Void subMl_KeyUp(System::Object^ sender,
   System::Windows::Forms::KeyEventArgs^ e) {
2          String^ text = subMl->Text;
3
4          try {
5                  int num = Convert::ToInt32(text);
6
7                  if (num <= 0) {
8                          MessageBox::Show("Wprowadź liczbę całkowitą
   dodatnią", "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Error);
9                          subMl->Text = "";
10                 }
11
12         }
13         catch (...) {
14                 MessageBox::Show("Wprowadź liczbę", "Program kalkulator",
   MessageBoxButtons::OK, MessageBoxIcon::Error);
15
16                 if (text->Length > 0) subMl->Text = text->Substring(0, text-
   >Length - 1);
17                 else subMl->Text = "";
18         }
19 }
20
```

W wewnątrz zakresu *try* sprawdzane jest czy wprowadzoną wartość można przekształcić na liczbę całkowitą. Jeśli tak, sprawdzane jest czy liczba jest większa od zera – w przeciwnym wypadku wyświetlany jest komunikat o konieczności zmiany wartości. W zakresie *catch* usuwany jest ostatni wprowadzony znak do etykiety oraz wyświetlana jest informacja o konieczności wprowadzania jedynie cyfr.

Pozostaje zadbać o zresetowanie tekstu zachęty dla kolejnych wywołań opcji dolewania substancji do kubka:

```
1  private: System::Void addSubMenuItem_Click(System::Object^ sender,
   System::EventArgs^ e) {
2          fillSubList();
3          subList->Location = System::Drawing::Point(12 + (10 + 199) * cupID,
   273);
4          subMl->Location = System::Drawing::Point(12 + (10 + 199) * cupID,
   302);
5          wlej->Location = System::Drawing::Point(12 + (10 + 199) * cupID, 330);
6          anuluj->Location = System::Drawing::Point(135 + (10 + 199) * cupID,
   330);
7
8          subMl->Text = L"Wpisz ilość w [ml]";
9
10         subList->Visible = true;
11         subMl->Visible = true;
12         wlej->Visible = true;
13         anuluj->Visible = true;
14 }
15
```

Uruchom program i sprawdź jak działa uzupełnianie ilości ml wlewanej substancji do kubka.

Pozostaje obsłużyć przyciski. W przypadku „Anuluj” sprawa jest prosta – kliknij dwukrotnie LPM na kontrolkę i do wygenerowanego szkieletu funkcji wstaw kod:

```
1  private: System::Void anuluj_Click(System::Object^ sender, System::EventArgs^
   e) {
2          cleanLblCup();
3  }
4
```

Powtórz akcję dla przycisku „Wlej” – tutaj kod będzie dłuższy o ponowne sprawdzenie poprawności wprowadzonych danych:

```
1  private: System::Void wlej_Click(System::Object^ sender, System::EventArgs^ e)
2  {
3          String^ text = subMl->Text;
4          try {
5                  int num = Convert::ToInt32(text);
6                  int index = subList->SelectedIndex;
7
8                  if (index >= 0) {
9                          MessageBox::Show("Poprawne dane - w kolejnym zadaniu
   zajmiemy się warstwą logiczną", "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Information);
11                         cleanLblCup();
12                 }
13                 else {
14                         MessageBox::Show("Wybierz ciecz do dolania!",
                                   "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Error);
15                 }
16         }
17         catch (...) {
18                 MessageBox::Show("Wprowadź liczbę", "Program kalkulator",
   MessageBoxButtons::OK, MessageBoxIcon::Error);
19                 subMl->Text = "";
20         }
21 }
22
```

W funkcji `cleanLblCup` musimy jeszcze ukryć nowe kontrolki po kliknięciu jednego z przycisków: „wlej” lub „anuluj”:

```
subList->Visible = false;
subMl->Visible = false;
wlej->Visible = false;
anuluj->Visible = false;

cupID = -1;

```

Sprawdź działanie programu!

Prześlij zmiany na repozytorium zdalne.

ZADANIE 2.2 – zamrożenie okna programu przy dodawaniu substancji do kubka

Zanim przejdziemy do obiecanego dodawania substancji do kubka w warstwie logicznej programu, zajmijmy się jeszcze jedną kwestią… Dosyć uciążliwą sprawą jest wyjście z opcji dodawania substancji do kubka w przypadku omyłkowego trafienia w tło okna programu zamiast listę substancji lub okno do wpisywania wartości cieczy w ml. Pewnym rozwiązaniem jest zablokowanie wszystkich innych funkcjonalności programu dopóki użytkownik zatwierdzi lub anuluje dodawanie substancji.

Na początek dodajmy zmienną pomocniczą informującą czy jesteśmy w stanie dodawania substancji do kubka:

```
#include <vector>
#include "OAutorze.h"
#include "TCup.h"

#pragma once

namespace POiIOkubeczki {

        using namespace System;
        using namespace System::ComponentModel;
        using namespace System::Collections;
        using namespace System::Windows::Forms;
        using namespace System::Data;
        using namespace System::Drawing;

        /// <summary>
        /// Podsumowanie informacji o MainWin
        /// </summary>
        public ref class MainWin : public System::Windows::Forms::Form
        {
        private:
                Generic::List<PictureBox^>^ cups = gcnew Generic::List<PictureBox^>();
                Generic::List<Label^>^ lbl_cups = gcnew Generic::List<Label^>();
                int cupID = -1;
                bool add_substance_active = false;

```

Zmienna przyjmuje wartość *true* w momencie kliknięcia opcji w menu „Dodaj substancję do kubka…”. Podczas aktywacji wspomnianej opcji możemy również deaktywować menu:

```
1  private: System::Void addSubMenuItem_Click(System::Object^ sender,
2  System::EventArgs^ e) {
3          add_substance_active = true;
4          menuStrip1->Enabled = false;
5          fillSubList();
6  ...
```

Ponowna zmiana wartości zmiennej na *false*, jak również aktywowanie menu, odbywa się w trakcie anulowania akcji dodawania substancji lub kliknięcia przycisku „wlej” przy poprawnie wprowadzonych danych.

Kod dla przycisku „anuluj”:

```
1  private: System::Void anuluj_Click(System::Object^ sender, System::EventArgs^
2  e) {
3          cleanLblCup();
4          add_substance_active = false;
5          menuStrip1->Enabled = true;
6  }
7
```

Kod dla przycisku „wlej”:

```
1  private: System::Void wlej_Click(System::Object^ sender, System::EventArgs^ e)
2  {
3          String^ text = subMl->Text;
4
5          try {
6                  int num = Convert::ToInt32(text);
7                  int index = subList->SelectedIndex;
8
9                  if (index >= 0) {
10                         MessageBox::Show("Poprawne dane - w kolejnym zadaniu
   zajmiemy się warstwą logiczną", "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Information);
11                         cleanLblCup();
12                         add_substance_active = false;
13                         menuStrip1->Enabled = true;
14                 }
15                 else {
16                         MessageBox::Show("Wybierz ciecz do dolania!",
                                   "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Error);
17                 }
18
19         }
20         catch (...) {
21                 MessageBox::Show("Wprowadź liczbę", "Program kalkulator",
   MessageBoxButtons::OK, MessageBoxIcon::Error);
22                 subMl->Text = "";
23         }
24 }
25
```

Pozostaje zamrozić pozostaje funkcjonalności programu w momencie, gdy zmienna *add_substance_active* przyjmuję wartość *true*.

Deaktywowanie kubka jest możliwe jedynie, gdy nie jesteśmy w stanie dodawania substancji:

```
private: System::Void MainWin_Click(System::Object^ sender, System::EventArgs^ e) {
        if (add_substance_active == false) cleanLblCup();
}
```

Podobnie wybór innego kubka – możliwy jedynie, gdy nie jesteśmy w stanie dodawania substancji::

```
private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e)
{
        if (add_substance_active == false)
        {
                cleanLblCup();

                Label^ lbl = (Label^)sender;
                lbl->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
                lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
                                System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
                lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);

                getCupID(lbl);
                delCupMenuItem->Enabled = true;
                delCupMenuItem->Text = L"Usuń kubek #" + Convert::ToString(cupID);
                addSubMenuItem->Enabled = true;
                addSubMenuItem->Text = L"Dodaj substancje do kubka #" + Convert::ToString(cupID);

                cups[cupID]->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
        }
}

```

Sprawdź działanie programu!

Prześlij zmiany na repozytorium zdalne.

ZADANIE 2.3 – dodawanie substancji do kubka: backend

Aktualnie zajmiemy się rzeczywistym dodaniem substancji do wybranego obiektu klasy *TCup*. Zmieniamy kod realizowany po kliknięciu przycisku „Wlej”, po poprawnym wprowadzeniu danych:

```
1  private: System::Void wlej_Click(System::Object^ sender, System::EventArgs^ e)
2  {
3          String^ text = subMl->Text;
4          int vol = 0;
5
6          try {
7                  vol = Convert::ToInt32(text);
8                  int index = subList->SelectedIndex;
9
10                 if (index >= 0) {
11                         add_substance_to_cup(vol);
12
13                         cleanLblCup();
14                         add_substance_active = false;
15                         menuStrip1->Enabled = true;
16                         //MessageBox::Show("Poprawne dane - w kolejnym
   zadaniu zajmiemy się warstwą logiczną",
17                         //      "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Information);
18
19                 }
20                 else {
21                         MessageBox::Show("Wybierz ciecz do dolania!",
                                   "Program kalkulator", MessageBoxButtons::OK,
   MessageBoxIcon::Error);
22                 }
23
24         }
25         catch (...) {
26                 MessageBox::Show("Wprowadź liczbę!!!", "Program kalkulator",
   MessageBoxButtons::OK, MessageBoxIcon::Error);
27                 subMl->Text = "";
28         }
29
30 }
31
```

Zakomentowane zostało polecenie wyświetlenia okna informacyjnego. U góry bloku instrukcji warunkowej dodano wywołanie nowej funkcji `add_substance_to_cup(vol)`. Zobaczmy kod nowej funkcji:

```
1  void add_substance_to_cup(int vol)
2  {
3          String^ selected = subList->SelectedItem->ToString();
4          std::string name = msclr::interop::marshal_as<std::string>(selected);
5
6          TCup* cup_pnt = cups_pnt[cupID];
7          cup_pnt->add(name, vol);
8          show_cup_info();
9  }
10
```

W linijce 3-4 dochodzi do przekształcenia zmiennej typu *String*^ (zmienna języka C++/CLI) na zmienną typu *string* (zmienna języka C++). Wywołanie polecenia z linii 4 wymaga dołączenia nowej biblioteki do pliku:

```
1  #include <msclr/marshal_cppstd.h>
2  #include <vector>
3  #include "OAutorze.h"
4
```

Następnie, w funkcji `add_substance_to_cup(vol)`, pobierany jest adres obiektu typu *TCup* i wywoływana jest jego wewnętrzna funkcja `add(name, vol)` do dolania substancji.

Poprawne działanie programu wymaga zmiany w funkcji `private: Void addTCup()`:

```
1  private: Void addTCup() {
2          TCup* cup = new TCup;
3          cups_pnt.push_back(cup);
4  }
5
```

Obiekt klasy *TCup* musi być tworzony **dynamicznie** za pomocą polecenia `new`. Prawa strona równania w linijce 2, tj. `new TCup`, powoduje dynamiczne utworzenie obiektu i zwrócenie jego adresu – przypisanego do wskaźnika o nazwie `cup`.

Dynamicznie utworzone obiekty nie są usuwane automatycznie, w tym przypadku po wyjściu z funkcji `addTCup()`. To użytkownik musi zadbać w tym przypadku o zwolnienie pamięci poprzez usunięcie obiektu poleceniem:

```
delete cup;
```

Ostatecznie, należy zwrócić uwagę że dla dynamicznie utworzonego obiektu dysponujemy jedynie wskaźnikiem do niego. W przypadku nadpisania wartości wskaźnika obiekt będzie istniał dalej, a my nie będziemy mieli już możliwości się do niego odwołać lub **go usunąć** – taką sytuację nazywa się wyciekiem pamięci [*memory leak*].

Wyciek pamięci występuje, gdy program przydziela pamięć na stercie, ale nie zwalnia jej, gdy nie jest już potrzebna. W rezultacie ta część pamięci staje się niedostępna dla systemu, a program zużywa coraz więcej zasobów, co może prowadzić do spowolnienia lub awarii systemu.

Finalnie, na końcu funkcji `add_substance_to_cup(vol)`, wywoływana jest kolejna nowa funkcja `show_cup_info()`. Jej celem jest wyświetlenie informacji o obecnej zawartości kubka, po dodaniu substancji, w oknie informacyjnym aplikacji (nie mamy dostępu do konsoli):

```
1  void show_cup_info()
2  {
3          TCup* cup_pnt = cups_pnt[cupID];
4          std::string info = cup_pnt->info();
5
6          String^ info_cli = gcnew String(info.c_str());
7          MessageBox::Show(info_cli, "Program kalkulator",
   MessageBoxButtons::OK, MessageBoxIcon::Information);
8  }
9
```

Dla klasy *TCup* implementujemy nową funkcję `info()`.

Zmiany w pliku nagłówkowym *TCup.h*:

```
1   class TCup
2   {
3           std::vector<TSubstance> substances;
4           std::vector<double> volumes;
5
6   public:
7           void add(TSubstance substance, double volume_in_ml);
8           void add(std::string name, double volume_in_ml);
9           void show();
10          std::string info(int _id);
11
12  private:
13          int get_substance_id(std::vector<TSubstance>* substances_buffer,
    std::string name);
14  };
15
16  extern std::vector<TCup*> cups_pnt;
17
```

Zmiany w pliku źródłowym *TCup.cpp* - kod nowej funkcji:

```
1   string TCup::info(int _id)
2   {
3           string info = "Kubek #" + to_string(_id) + "\n";
4           int count = substances.size();
5
6           for (int i = 0; i < count; i++)
7           {
8                   string ml = std::to_string(volumes[i] * 1e6);
9                   info += substances[i].get_name() + ": " + ml + "ml\n";
10          }
11
12          return info;
13  }
14
```

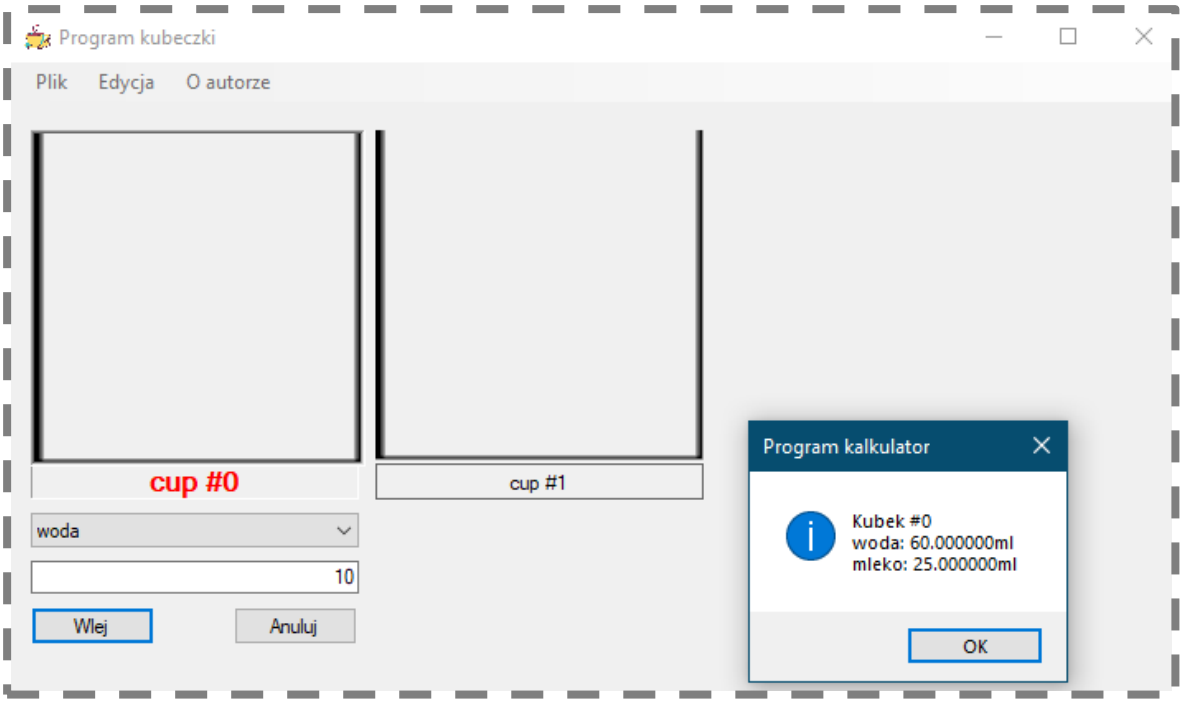
Konwersja zmiennej typu *double* na *string* wymaga dołączenia biblioteki na początku pliku *TCup.cpp*:

```
1   #include <typeinfo>
2   #include <string>
3   #include "TCup.h"
4   using namespace std;
5
```

Sprawdź działanie programu!

Prześlij zmiany na repozytorium zdalne.

Przykład działania programu:



ZADANIE 2.4 – dodawanie substancji do kubka: GUI

Zaimplementujemy i wywołamy dwie dodatkowe funkcje do rysowania obecności substancji w kubku:

```
1   private: System::Void wlej_Click(System::Object^ sender, System::EventArgs^ e)
2   {
3           String^ text = subMl->Text;
4           int vol = 0;
5
6           try {
7                   vol = Convert::ToInt32(text);
8                   int index = subList->SelectedIndex;
9
10                  if (index >= 0) {
11                          add_substance_to_cup(vol);
12                          erase_cup_substances();
13                          draw_cup_substances();
14
15                          cleanLblCup();
16                          add_substance_active = false;
17                          menuStrip1->Enabled = true;
18                  }
19                  else {
20                          MessageBox::Show("Wybierz ciecz do dolania!",
                                    "Program kalkulator", MessageBoxButtons::OK,
    MessageBoxIcon::Error);
21                  }
22          }
23          catch (...) {
24                  MessageBox::Show("Wprowadź liczbę!!!", "Program kalkulator",
    MessageBoxButtons::OK, MessageBoxIcon::Error);
25                  subMl->Text = "";
26          }
27  }
28
```

W pierwszej kolejności przyjrzymy się funkcji `draw_cup_substances`:

```
1   void draw_cup_substances()
2   {
3           TCup* cup_pnt = cups_pnt[cupID];
4           std::vector<TSubstance> subs = cup_pnt->get_cup_substances();
5           std::vector<double> vols = cup_pnt->get_cup_volumes();
6
7           int vol_in_cup = 0;
8
9           for (int i = 0; i < vols.size(); i++)
10          {
11                  TSubstance sub = subs[i];
12                  std::string name = sub.get_name();
13                  int vol = vols[i] * 1e6;
14                  std::vector<int> color_rgb = sub.get_color();
15                  String^ name_cli = gcnew String(name.c_str());
16                  name_cli += L": " + Convert::ToString(vol) + L"ml";
17
18                  Label^ cup_substance = (gcnew
    System::Windows::Forms::Label());
19                  cup_substance->BorderStyle =
    System::Windows::Forms::BorderStyle::FixedSingle;
20                  cup_substance->TextAlign =
    System::Drawing::ContentAlignment::MiddleCenter;
21
22                  cup_substance->Size = System::Drawing::Size(199-12, vol);
23                  cup_substance->Location = System::Drawing::Point(12+6 + (10 +
    199) * cupID, 238 - vol_in_cup - vol);
24                  cup_substance->Tag = Convert::ToString(cupID);
25                  cup_substance->Text = name_cli;
26                  cup_substance->BackColor = Color::FromArgb(color_rgb[0],
    color_rgb[1], color_rgb[2]);
27                  cup_substance->ForeColor = Color::FromArgb(255-color_rgb[0],
    255-color_rgb[1], 255-color_rgb[2]);
28
29                  cup_substance->BringToFront();
30                  this->Controls->Add(cup_substance);
31                  lbl_subs->Add(cup_substance);
32                  vol_in_cup += vol;
33          }
34
35          cups[cupID]->SendToBack();
36  }
37
```

W pierwszych linijkach odczytujemy informacje o substancjach w aktywnym kubku. Następnie w pętli for (linie 9-33):

- Odczytujemy informacje o nazwie, objętości i kolorze substancji, tworzymy zmienną typu *String^*, która wyświetlać będzie informacje o nazwie i objętości cieczy.

- Tworzymy kontrolkę typu ***Label***.

- Nadajemy właściwości charakterystyczne dla substancji i kubka: określamy pozycję substancji w kubku, nadajemy wyświetlaną nazwę i definiujemy kolory tła i tekstu. Zwróć uwagę na linię 24 – tworzymy zmienne pomocniczą wewnątrz kontrolki z informacją do którego kubka wlano substancję, będziemy korzystać z tej informacji przy aktualizacji zawartości kubka.

- Dodajemy nową kontrolkę do zestawu Controls i listy kontrolek odpowiadających wyświetlaniu cieczy w aplikacji. W linii 29 powodujemy wyświetlenie kontrolki przed obrazkiem kubka.

Mogłeś zauważyć, że potrzebujemy nowych funkcji dla klasy *TCup*:

```
 1  class TCup
 2  {
 3          std::vector<TSubstance> substances;
 4          std::vector<double> volumes;
 5
 6  public:
 7          void add(TSubstance substance, double volume_in_ml);
 8          void add(std::string name, double volume_in_ml);
 9          void show();
10          std::string info(int _id);
11
12          std::vector<TSubstance> get_cup_substances() {
13                  return substances;
14          }
15
16          std::vector<double> get_cup_volumes() {
17                  return volumes;
18          }
19
20  private:
21          int get_substance_id(std::vector<TSubstance>* substances_buffer,
22  std::string name);
23  };
24
```

Podobnie dla klasy *TSubstance* dodaliśmy nową cechę – kolor substancji:

TSubstance.h

```
 1  #pragma once
 2  #include <iostream>
 3  #include <vector>
 4
 5  class TSubstance
 6  {
 7          std::string name;
 8          double ro; //[kg/m3]
 9          std::vector<int> color = {0, 0, 0};
10
11  public:
12          TSubstance(std::string name, double ro, int red, int green, int blue);
13
14          std::string get_name();
15          double get_ro();
16          std::vector<int> get_color();
17  };
18
19  extern std::vector<TSubstance> substance_menu;
20
```

TSubstance.cpp

```
1  #include "TSubstance.h"
2  using namespace std;
3
4  TSubstance::TSubstance(string name, double ro, int red, int green, int blue)
5  {
6          this->name = name;
7          this->ro = ro;
8          this->color[0] = red;
9          this->color[1] = green;
10         this->color[2] = blue;
11 }
12
13 string TSubstance::get_name()
14 {
15         return name;
16 }
17
18 double TSubstance::get_ro()
19 {
20         return ro;
21 }
22
23 std::vector<int> TSubstance::get_color()
24 {
25         return color;
26 }
27
28
29 vector<TSubstance> substance_menu =
30 {
31                 TSubstance{"woda", 997.8, 95, 165, 250},
32                 TSubstance{"mleko", 1030.0, 192, 230, 238},
33                 TSubstance{"oliwa", 920.0, 55, 115, 78},
34                 TSubstance{"krew", 1050.0, 241, 87, 87},
35                 TSubstance{"ocet", 1050.0, 235, 243, 156}
36 };
37
```

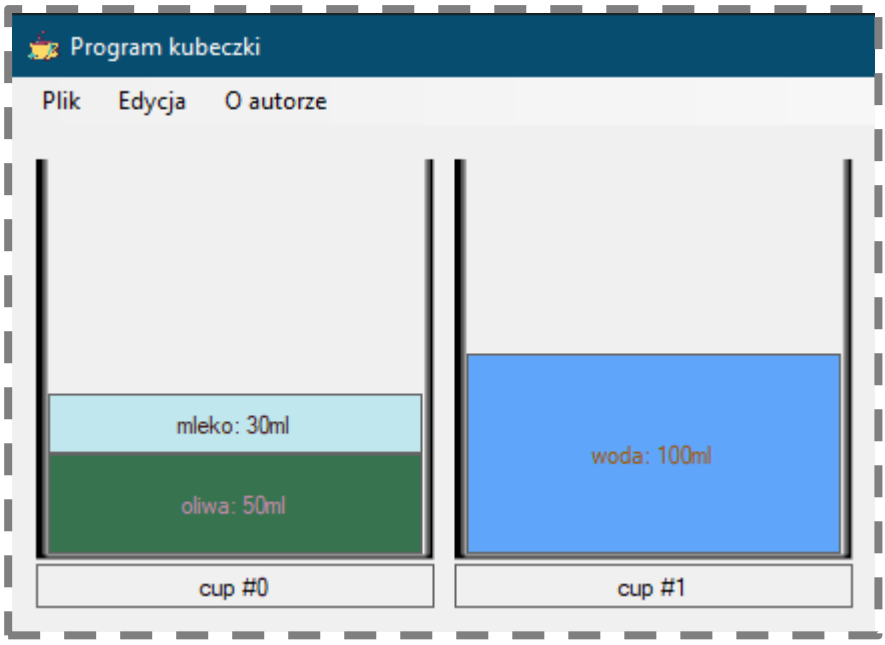
Teoretycznie powyższe zmiany wystarczyłyby do wyświetlenia substancji w kubku. Mając jednak na uwadze możliwość zmiany objętości substancji już istniejącej w kubku, musimy obsłużyć tą aktualizację. Najprostszą metodą będzie usunięcie wszystkich ***Label*** przypisanych do aktywnego kubka i ponowne ich utworzenie za pomocą omówionej już funkcji `draw_cup_substances`.

Za usunięcie obecnych kontrolek substancji w aktywnym kubku odpowiada funkcja `erase_cup_substances`:

```
1  void erase_cup_substances()
2  {
3          for each(Label ^ sub in lbl_subs)
4          {
5                  if (sub && (Convert::ToInt16(sub->Tag) == cupID))
6                  {
7                          Controls->Remove(sub);
8                          sub = nullptr;
9                          delete sub;
10                 }
11         }
12 }
13
```

Sprawdź działanie programu!

Przykładowy wynik:



Prześlij zmiany na repozytorium zdalne.

ZADANIE 2.5 – skróty klawiszowe

W celu przyśpieszenia korzystania z programu dodamy sobie skróty klawiszowe. Chcemy mieć możliwość dodania kubka, dodania substancji do aktywnego kubka i wybór aktywnego kubka. Skorzystamy z klawiszy:

- Wciśnięcie „+” przy nieaktywnym kubku spowoduje dodanie nowego kubka.
- Wciśnięcie „+” przy aktywnym kubku spowoduje dodawanie substancji.
- Wciśnięcie przycisków numerycznych (0-9) spowoduje wybór aktywnego kubka o odpowiednim ID – przy założeniu, że nie jesteśmy w trakcie dodawania substancji.

Na wstępie musimy wyciąć kod odpowiednich zdarzeń do nowych funkcji – tak, aby można było je wywołać z dwóch miejsc, tj. albo przez wybór w menu albo wciśnięcie klawisza skrótu.

Dla dodania kubka:

```
1  private: System::Void dodajKubekToolStripMenuItem_Click(System::Object^
   sender, System::EventArgs^ e) {
2          add_cup_gui();
3  }
4
5  private: Void add_cup_gui()
6  {
7          addTCup();
8          addCup();
9          addLblCup();
10 }
11
```

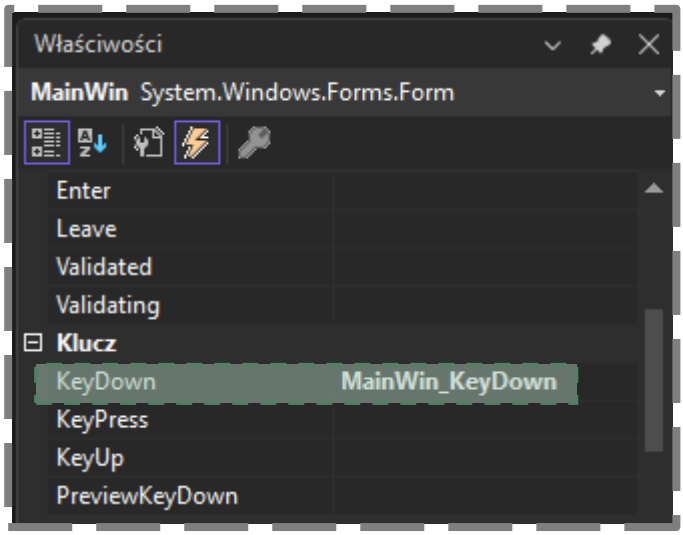
Dla dodawania substancji:

```
1  private: System::Void addSubMenuItem_Click(System::Object^ sender,
   System::EventArgs^ e) {
2          add_sub_gui();
3  }
4
5  private: Void add_sub_gui()
6  {
7          add_substance_active = true;
8          menuStrip1->Enabled = false;
9          fillSubList();
10         subList->Location = System::Drawing::Point(12 + (10 + 199) * cupID,
   273);
11         subMl->Location = System::Drawing::Point(12 + (10 + 199) * cupID,
   302);
12         wlej->Location = System::Drawing::Point(12 + (10 + 199) * cupID, 330);
13         anuluj->Location = System::Drawing::Point(135 + (10 + 199) * cupID,
   330);
14
15         subMl->Text = L"Wpisz ilość w [ml]";
16
17         subList->Visible = true;
18         subMl->Visible = true;
19         wlej->Visible = true;
20         anuluj->Visible = true;
21 }
22
```

Dla wyboru aktywnego kubka:

```
private: System::Void selectCup(System::Object^ sender, System::EventArgs^ e)
{
        select_cup_gui(sender);
}

private: Void select_cup_gui(System::Object^ sender)
{
        if (add_substance_active == false)
        {
                cleanLblCup();

                Label^ lbl = (Label^)sender;
                lbl->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
                lbl->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 12, System::Drawing::FontStyle::Bold,
                                System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(238)));
                lbl->ForeColor = System::Drawing::Color::FromArgb(255, 0, 0);

                getCupID(lbl);
                delCupMenuItem->Enabled = true;
                delCupMenuItem->Text = L"Usuń kubek #" + Convert::ToString(cupID);
                addSubMenuItem->Enabled = true;
                addSubMenuItem->Text = L"Dodaj substancje do kubka #" + Convert::ToString(cupID);

                cups[cupID]->BorderStyle = System::Windows::Forms::BorderStyle::Fixed3D;
        }
}

```

W IDE przejdź do zakładki „*MainWin.h [Projekt]*”, kliknij raz LPM na tło aplikacji i w oknie ***Właściwości*** kliknij dwukrotnie LPM na zdarzenie ***KeyDown*** – zostanie wygenerowania funkcja obsługująca zdarzenie, zostaniesz przekierowany do pustego ciała funkcji.



Kod funkcji:

```
1  private: System::Void MainWin_KeyDown(System::Object^ sender,
   System::Windows::Forms::KeyEventArgs^ e) {
2          if (e->KeyCode == Keys::Add)
3          {
4                  if (cupID == -1) add_cup_gui();
5                  else add_sub_gui();
6          }
7
8          if ((e->KeyCode >= Keys::D0) && (e->KeyCode <= Keys::D9))
9          {
10                 int i = e->KeyValue - 48;
11
12                 if (i < cups->Count)
13                 {
14                         Label^ cup_lbl = lbl_cups[i];
15                         select_cup_gui(cup_lbl);
16                 }
17         }
18 }
19
```

Zwróć uwagę, że w linii 10 skorzystałem z właściwości *KeyValue* w celu odczytu, który klawisz numeryczny został wciśnięty. Podobnie jak w ASCII wartości numeryczne posortowane są rosnąco, co ułatwia odczytanie wartości klawisza. Musimy jednak znać wartość kodu dla „0”. Wartości dla *KeyValue* znajdziesz na oficjalnej stronie: link.

Domyślnie informacja o wciśniętym klawiszu zostaje przesłana do aktywnego elementu GUI. W przypadku dynamicznego tworzenia kontrolek, ostatni utworzony element będzie tym aktywnym… W celu zawsze sprawdzania informacji o wciśniętym przycisku przez funkcję powiązaną z formą, w konstruktorze aplikacji należy dodać polecenie:

```
40 public:
41         MainWin(void)
42         {
43                 InitializeComponent();
44                 this->KeyPreview = true;
45                 //
46                 //TODO: W tym miejscu dodaj kod konstruktora
47                 //
48         }
49
```

Sprawdź działanie programu!

Prześlij zmiany na repozytorium zdalne.

3. Podsumowanie

W niniejszej instrukcji dokończyliśmy podstawową funkcjonalność programu. Najistotniejszą kwestią było połączenie interfejsu użytkownika z warstwą logiczną programu, napisaną podczas wcześniejszych zajęć. Bardzo istotną kwestią jest rozdzielenie *frontendu* aplikacji od jej *backendu*. Dzięki takiemu podejściu mam zachowaną odpowiednią architekturę aplikacji. W przypadku zmiany podejścia, np. dalszego rozwoju projektu jako aplikacji webowej, *backend* jest gotowy do użycia w nowej wersji projektu. Poza tym, często inne zespoły będą odpowiedzialne za te dwie części projektu.

Zwróć również uwagę na dodatkowy nakład pracy konieczny do utworzenia graficznego interfejsu użytkownika. Dobrym pomysłem we wstępnej fazie projektu jest skupienie się jedynie na *backend'zie*, warstwie logicznej programu. Podobnie jak zrobiliśmy to na zajęciach. Jest to również naturalna ochrona przed mieszaniem kodu z warstwy logicznej z *frontend'em*.